

SAHAYAK AI

Multilingual Cooperative Governance & Legal Assistance Chatbot

Detailed Prototype Implementation Plan

A development-ready blueprint for a hardware-enabled, voice-first rural assistance kiosk with a mobile-phone handoff for complex tasks.

Prototype principle	The chatbot box should solve common user problems by itself. The user's personal phone is used only when a task genuinely needs a larger interface, camera, personal information, or complex interaction.
Core interaction	User approaches kiosk → says “Hello Sahayak” → speaks in a supported language → Sahayak answers by voice and/or screen → QR handoff only when needed.
Primary device	A smartphone embedded inside the physical chatbot box provides the screen, microphone, speaker, camera if needed, network connection, and main kiosk UI.
External device	The user's own phone becomes an extension of Sahayak for complex visual tasks, document/photo capture, long-form interaction, and personal dashboard access.

Prepared as a handoff document for an AI development agent / software-hardware prototype team.

1. Problem Definition

The problem statement describes rural cooperative members, farmers, and other rural stakeholders who may struggle to access reliable guidance because of language barriers and limited access to clear information about cooperative laws, government schemes, PACS services, crop insurance, financial literacy, and grievance redressal. The requested system is a multilingual AI chatbot with voice assistance and mobile/web integration.

The prototype therefore must not behave like a generic FAQ chatbot. It should be designed around a real-world task: understand the user's problem, explain the relevant information simply, and tell the user what to do next.

Prototype goals

- Voice-first interaction so a user does not need to type.
- Simple multilingual conversation with language switching.
- Reliable, source-backed answers from an approved knowledge base.
- Step-by-step action plans rather than only explanatory paragraphs.
- Guidance for cooperative governance, schemes/services, agricultural support, financial literacy, and grievances.
- A hardware kiosk that can independently handle many common questions.
- A QR-based handoff to the user's own phone only for complex tasks.
- Document/image understanding through the user's phone when a personal camera is required.
- A persistent conversation/session so the user can continue after QR handoff.
- Clear separation between informational/legal guidance and professional/legal authority.

Source scope for the prototype

Use the uploaded problem statement as the source of the required feature scope. Do not claim that a scheme, law, eligibility rule, contact number, or procedure is current unless it has been verified from the relevant official source before being loaded into the knowledge base.

2. Overall System Architecture

Recommended high-level architecture:

User → Embedded Smartphone Kiosk UI → Voice/Intent Layer → AI Orchestrator → Verified Knowledge Base/RAG → Response + Action Plan → Voice/Screen

For complex work:

Kiosk → Generate Session/QR → User scans QR → Mobile Sahayak Dashboard → Continue session → Camera/upload/forms/long answer → Backend → Kiosk can optionally reflect the updated session status.

Architecture layers

Layer	Responsibilities
Physical kiosk	Box/enclosure, embedded smartphone mount, power, optional status LED/button.
Embedded smartphone	Touch UI, microphone, speaker, wake-word detection, conversation UI, QR display, network connection.
Frontend kiosk app	Simple chatbot UI, language selector, status states, transcript, QR handoff, session state.
Backend API	Authentication/session management, conversation orchestration, document processing, RAG, action plans, QR t
AI layer	Intent classification, retrieval, answer generation, translation/simplification, STT/TTS integration, OCR/image an
Knowledge layer	Approved cooperative/legal/scheme/financial/grievance documents, metadata, source URLs, effective dates, ve
Vector search	Semantic retrieval of relevant passages from approved documents.
Database	Users/sessions, messages, intents, documents, source metadata, action plans, QR sessions, analytics.
Mobile dashboard	Complex continuation, image capture/upload, document scanning, long-form results, saved action plan.
Admin console	Upload/update documents, review extracted text, approve content, version sources, monitor system health.

Important implementation decision

Do not build separate AI logic for kiosk and mobile. Both clients should use the same backend session and AI orchestration layer. The kiosk and mobile dashboard are two interfaces to the same Sahayak session.

3. Hardware Prototype Plan

The hardware should be deliberately simple. The embedded smartphone is the main computing and I/O device; this removes the need for Raspberry Pi or a dedicated PC in the physical prototype.

Component	Role	Priority
Android smartphone	Embedded display, touch input, microphone, speaker, camera, network, QR display, app runtime.	Mandatory
Stable power adapter	Continuous power to the embedded smartphone.	Mandatory
Kiosk enclosure	Holds the smartphone and makes the device look like a dedicated public-service assistant.	Mandatory
Optional physical wake/help button	Alternative interaction trigger and accessibility backup.	Recommended
Optional LED/RGB indicator	Shows listening, processing, speaking, error states.	Recommended
ESP32 (optional)	Provides physical buttons/LED control and demonstrates hardware-software integration.	Optional
Document stand	Positions papers in front of the phone camera for scan demo.	Recommended
Thermal printer	Prints short checklists/ticket-like action summaries if desired.	Optional
Power bank/UPS	Backup power for demonstrations.	Optional

Physical design

- Mount the smartphone vertically in portrait orientation.
- Keep the screen and front speaker/microphone unobstructed.
- Create a small opening around the phone camera if camera capture is demonstrated from the kiosk.
- Add a simple face/robot/chatbot visual around the phone so the phone feels like the 'face' of Sahayak.
- Use a clear front area with the text 'Say: Hello Sahayak'.
- Place a document stand below or beside the phone for scan demonstrations.
- If using ESP32, place a large HELP or TALK button where a first-time user can see it.
- Avoid unnecessary sensors and biometric hardware in the first prototype.

Hardware states

State	Visual/Audio behavior
Idle	Chatbot emoji gently animated; text: 'Hello, I am Sahayak. Say Hello Sahayak to start.'
Wake detected	Emoji changes state; short chime; text: 'Listening...'
Listening	Animated microphone/listening indicator.
Processing	Spinner/animated chatbot face; no misleading 'typing' delay.
Speaking	Speaker icon/animated mouth; screen shows short answer.
QR handoff	Large QR code + short instruction: 'Scan to continue on your phone.'
Error/offline	Simple message and retry action; never expose technical stack traces.

4. Kiosk UX and Interaction Design

The kiosk UI should be intentionally minimal. The user should understand what to do within a few seconds without reading a long instruction screen.

Home screen

- Large chatbot emoji/character in the center.
- Greeting: 'Namaste! Main Sahayak hoon.' or equivalent in selected language.
- Primary instruction: 'Say Hello Sahayak to start.'
- Small language button.
- Small HELP button.
- Optional animated listening/mouth state.

Wake-word flow

Hello Sahayak → wake word detector activates → short acknowledgement → microphone listens → STT converts speech to text → intent detection → answer generation → TTS speaks response.

Speech-only answer policy

If the response can be understood naturally through a short spoken explanation, the kiosk should primarily speak it. The screen may show a concise supporting summary, but the user should not be forced to read.

Examples of speech-only or speech-first tasks

- What is a cooperative society?
- What is PACS?
- What is crop insurance?
- Explain a basic financial term.
- What is a grievance?
- What documents are generally needed for a specified service when the answer is short and verified.
- Simple definitions and basic procedural guidance.

Complex-output detection

The system should classify whether the response requires a richer visual interface. Trigger QR handoff when the task involves long tables, many documents, image upload, camera capture, form completion, detailed comparison, large action plans, or any workflow that is awkward on the kiosk screen.

QR handoff flow

- 1 Backend creates a short-lived secure session token.
- 2 Kiosk requests a QR payload from the backend.
- 3 Kiosk renders the QR code full-screen.
- 4 User scans the QR with their personal phone.
- 5 Mobile web app opens the Sahayak dashboard and joins the existing session.
- 6 The dashboard displays conversation context so the user does not need to repeat the question.
- 7 The user completes the complex task.
- 8 Results are saved to the same session.

- 9 The kiosk can optionally show 'Continue on your phone' or a session-complete state.

5. Full Functional Feature Set

The following feature set combines the requested problem-statement functionality with the proposed differentiating design.

A. Multilingual Voice Assistant

- Wake phrase: 'Hello Sahayak'.
- Speech-to-text input.
- Text-to-speech output.
- Language selector.
- Language switching without losing conversation context.
- Simple-language mode so translation is accompanied by simplification.
- Voice-first UX for users who may have difficulty typing.

B. Intent Detection

- Classify requests into cooperative governance, legal/by-law information, government schemes, PACS services, agricultural support/insurance, financial literacy, grievance redressal, document explanation, general information, or unsupported request.
- Extract important entities such as scheme name, service, crop, problem, document type, state/district where appropriate.
- Ask only the minimum follow-up questions needed to provide useful guidance.

C. Verified RAG / Knowledge Assistant

- Use retrieval-augmented generation over an approved document set.
- Every substantive answer should be traceable to retrieved source passages.
- Store source title, issuing organization, URL, publication/effective date, last verification date, and document version.
- Use confidence/coverage checks before generating an answer.
- If evidence is insufficient, say that the information could not be verified rather than inventing an answer.

D. Scheme Navigator

- Search schemes/services by user problem.
- Show who the scheme/service is for, broad purpose, required documents, application route, and next steps where verified.
- Generate a personalized checklist only from verified rules.
- Allow user to switch from explanation to action plan.

E. Cooperative Governance / Legal Information

- Explain laws/by-laws and governance concepts in plain language.
- Identify relevant document sections through RAG.
- Provide source references.
- Clearly label the system as an information assistant rather than a substitute for an authorized legal professional or government officer.

- Avoid confident conclusions when the source material is ambiguous.

F. Agricultural / Insurance Guidance

- Answer verified questions about crop insurance/agricultural support.
- Use intent-driven questions to collect relevant context.
- Provide action steps and required documents when supported by the knowledge base.
- Avoid presenting stale deadlines or eligibility rules as current.

G. Financial Literacy

- Explain basic banking, credit, interest, savings, insurance, and cooperative-finance concepts in simple language.
- Use examples and spoken explanations.
- For calculations, route to a deterministic calculator rather than relying on LLM arithmetic.

H. Grievance Navigator

- Understand the user's complaint.
- Identify likely grievance category.
- Explain the documented escalation route.
- Generate a structured complaint draft.
- Generate a checklist of evidence/documents.
- Hand off to mobile for long form filling or image/document attachments.

6. Scan & Understand + Personal Phone Handoff

This is one of the strongest demonstrations of why the solution has hardware and software together.

Kiosk-only scan option

- If the embedded smartphone camera can clearly capture a document placed on the stand, allow a simple scan from the kiosk itself.
- OCR extracts text.
- RAG searches the approved knowledge base for relevant concepts.
- AI explains the document in the user's language.
- TTS reads a concise explanation aloud.

Personal-phone scan option

- If the document is in the user's hand or the kiosk camera cannot capture it reliably, show QR handoff.
- User scans QR and opens mobile dashboard.
- Mobile browser/app requests camera permission.
- User photographs the document.
- Backend receives the image securely.
- OCR extracts text and preserves page/region information where possible.
- Document understanding module identifies key fields/sections.
- RAG retrieves relevant verified guidance.
- Mobile displays detailed explanation; kiosk can provide a short spoken confirmation.

Complex-work routing rules

Task	Kiosk	Phone dashboard
Simple question	Solve directly	Not needed
Short definition	Solve directly	Not needed
Short action guidance	Solve directly	Optional
Long checklist	Give short summary + QR	Detailed view
Image/document upload	QR unless kiosk camera is suitable	Preferred
Multiple photos	QR	Preferred
Long form	QR	Preferred
Personal document details	QR	Preferred
Large comparison/table	QR	Preferred
Saved history/dashboard	QR	Preferred
Complex grievance submission	Start on kiosk, finish on phone	Preferred

7. Backend Implementation Plan

Recommended stack for a fast hackathon prototype

Area	Recommended option	Purpose
Backend	FastAPI + Python	REST/WebSocket APIs, AI orchestration, sessions.
Database	PostgreSQL	Structured session/user/document metadata.
Vector database	pgvector or a dedicated vector DB	Semantic document retrieval.
Object storage	S3-compatible storage or local prototype storage	Original PDFs/images/audio artifacts.
OCR	Cloud OCR or local OCR service	Document text extraction.
STT	Speech recognition API or local model	Convert voice to text.
TTS	Speech synthesis API or local engine	Spoken response.
LLM	API-based LLM for prototype	Reasoning, extraction, response generation.
Frontend kiosk	React / Next.js / Android WebView wrapper	Minimal kiosk UI.
Mobile	Responsive React web app / PWA	QR continuation without requiring app-store installation.
QR	Server-generated short-lived signed session URLs	Secure session handoff.
Admin	React admin panel + FastAPI	Knowledge-base updates and verification.

Suggested project structure

```
backend/  
  api/ — routes, authentication, sessions  
  core/ — configuration, security, logging  
  ai/ — orchestrator, prompts, intent classification, response policy  
  rag/ — ingestion, chunking, embeddings, retrieval, citations  
  speech/ — STT/TTS adapters  
  vision/ — OCR, image/document analysis  
  knowledge/ — document metadata and verification workflows  
  db/ — models, migrations, repositories  
  services/ — QR sessions, action plans, grievance workflows  
  tests/ — unit/integration tests  
  
frontend-kiosk/ — home, wake state, chat, language, QR handoff, accessibility  
frontend-mobile/ — session continuation, camera/upload, dashboard, action plan, history  
admin/ — document upload, review, approval, source/version management
```

8. Knowledge Base and RAG Pipeline

The quality of the prototype will depend heavily on the knowledge base. A generic LLM must not be treated as the source of truth for government schemes or legal procedures.

Document ingestion pipeline

- 1 Admin uploads an approved official PDF/web document.
- 2 System extracts text and document metadata.
- 3 OCR is used if the source is scanned.
- 4 Text is cleaned without destroying section headings or lists.
- 5 Document is split into semantically meaningful chunks.
- 6 Each chunk receives metadata: document ID, title, issuing body, section, page, effective date, verification date, language, status.
- 7 Embeddings are generated and stored.
- 8 Document remains in 'pending review' until approved.
- 9 Approved chunks become searchable by the RAG layer.
- 10 When a document is superseded, mark the old version accordingly rather than silently deleting it.

Recommended metadata

Field	Example
document_id	coop-law-001
title	Official document title
issuer	Issuing ministry/department
source_url	Official source URL
publication_date	YYYY-MM-DD if known
effective_from	YYYY-MM-DD if applicable
effective_to	YYYY-MM-DD or null
last_verified_at	YYYY-MM-DD
version	v1 / v2
language	Hindi / English / regional
status	pending / approved / superseded
page	PDF page number
section	Heading/section identifier

RAG answer pipeline

User query → language normalization → intent detection → entity extraction → query rewriting → vector/keyword retrieval → metadata filtering → evidence scoring → answer generation → source attachment → action-plan generation → response policy → TTS + screen rendering.

Anti-hallucination rules

- Do not answer policy/legal eligibility questions from model memory when no approved evidence was retrieved.

- Do not fabricate scheme names, deadlines, application links, phone numbers, fees, or authorities.
- Prefer 'I could not verify this from the available official information' to an unsupported answer.
- Show source information for substantive answers.
- Use deterministic templates for highly structured outputs such as document checklists where possible.

9. Database Design and Core APIs

Minimum database entities

Entity	Key fields
Session	id, kiosk_id, mobile_token, language, status, created_at, expires_at
Message	id, session_id, role, text, audio_ref, intent, created_at
Intent	id, name, confidence, extracted_entities
Document	id, title, issuer, source_url, version, status, verification dates
Chunk	id, document_id, text, page, section, embedding
SourceCitation	message_id, document_id, chunk_id, relevance
ActionPlan	session_id, title, steps, documents_required, warnings
QRSession	token_hash, session_id, expires_at, consumed_at
Grievance	session_id, category, complaint_text, draft, status
Upload	session_id, file_ref, type, OCR_text, processing_status
AuditLog	actor, action, object_type, object_id, timestamp

Core APIs

Method	Endpoint	Purpose
POST	/session	Create kiosk conversation session.
POST	/chat	Send text/intent request and receive response.
POST	/voice/transcribe	Convert uploaded audio to text.
POST	/voice/synthesize	Generate TTS audio or return provider reference.
POST	/qr/create	Create secure QR handoff for current session.
POST	/qr/join	Join an existing session from mobile.
POST	/upload	Upload document/image from mobile.
POST	/document/analyze	OCR + document understanding.
GET	/session/{id}	Fetch session context.
GET	/session/{id}/action-plan	Fetch current action plan.
POST	/grievance/draft	Generate structured complaint draft.
POST	/admin/documents	Upload knowledge document.
POST	/admin/documents/{id}/approve	Approve a reviewed document.

10. AI Orchestration and Decision Logic

The LLM should not directly control the whole application. Put deterministic application logic around it.

Recommended orchestration sequence

- 1 Receive text or STT transcript.
- 2 Detect language.
- 3 Normalize spelling/transliteration where necessary.
- 4 Classify intent.
- 5 Extract entities and missing parameters.
- 6 Check whether the request is supported.
- 7 Retrieve verified evidence.
- 8 Determine answer mode: SPEECH_ONLY, SCREEN_SUMMARY, or MOBILE_HANDOFF.
- 9 Generate answer from evidence.
- 10 Generate action plan if relevant.
- 11 Attach sources.
- 12 Convert response into simple language.
- 13 Send text to kiosk UI.
- 14 Send concise response to TTS.
- 15 If MOBILE_HANDOFF, generate secure QR and transition UI.

Response-mode policy

Mode	Trigger	Kiosk behavior
SPEECH_ONLY	Simple factual/short guidance answer.	Speak answer; show short summary.
SCREEN_SUMMARY	Answer is moderately detailed but still manageable.	Speak concise version + show structured bullets.
MOBILE_HANDOFF	Image upload, long form, complex table, multiple documents, long explanation, or need for workflow.	Explain why phone is needed and show QR.
HUMAN_ESCALATION	Insufficient evidence, high-risk/legal ambiguity, unresolved explanation, unsupported request.	Explain where to go for verified escalation path.

Example intent object

```
{
  "intent": "crop_insurance_guidance",
  "language": "hi",
  "entities": {"crop": "rice", "problem": "crop_damage"},
  "missing": ["location", "season"],
  "response_mode": "SCREEN_SUMMARY"
}
```

11. Wake Word and Voice Implementation

Wake phrase target: “Hello Sahayak”.

Implementation approach

- Run a lightweight wake-word detector locally on the embedded smartphone if the chosen platform supports reliable always-on listening.
- Do not continuously stream raw microphone audio to the server just to detect the wake phrase.
- After wake detection, start a short recording window or voice-activity-detection session.
- Stop recording when the user pauses for the configured duration.
- Send audio to STT.
- Show transcript confirmation on screen where useful.
- Generate response.
- Play TTS through the phone speaker.
- Return to idle after a timeout.

Fallback

Because wake-word reliability can vary across devices, include a visible TALK/HELP button in the prototype. The demo can show wake-word operation while the button provides a deterministic backup.

Speech safety

- Use a short acknowledgement so the user knows the system heard them.
- Handle background noise and silence gracefully.
- Never speak a fabricated answer just because STT confidence is low.
- If transcription is uncertain, ask the user to repeat the question.

12. Mobile Sahayak Dashboard

The mobile dashboard is not a separate product. It is the complex-work extension of the kiosk session.

Dashboard screens

Screen	Function
Session Join	Receives signed session token from QR and loads context.
Conversation	Full text conversation with voice/text input.
Action Plan	Step-by-step instructions and document checklist.
Scan Document	Camera capture, crop, multi-page scan, upload.
Analyze Image	Image explanation, extracted fields, relevant guidance.
Scheme Detail	Long-form verified information and source references.
Grievance	Complaint draft, evidence checklist, next-step workflow.
Sources	Documents, issuing body, dates, source links, relevant excerpts.
History	Previous sessions only if the prototype implements user identity.

PWA recommendation

For the hackathon, use a responsive PWA/web app rather than requiring users to install an Android application. QR → browser → session join is faster and easier to demonstrate.

Security requirements

- Use HTTPS in deployed environments.
- QR tokens must be random, short-lived, single-use or tightly scoped.
- Do not encode personal information directly into the QR.
- Expire abandoned sessions.
- Require explicit user action before uploading images or sensitive documents.
- Do not retain uploaded documents longer than necessary for the prototype.

13. Admin and Knowledge Management

A strong prototype should demonstrate how the system remains trustworthy after deployment. Build a simple admin workflow.

Admin workflow

Upload → Extract/OCR → Review → Add metadata → Approve → Embed/index → Publish → Version/verify → Supersede when replaced.

Admin features

- Login for admin.
- Upload PDF/document.
- Preview extracted text.
- Edit metadata.
- Set source URL and issuer.
- Set verification/effective dates.
- Approve/reject.
- Re-index approved document.
- Mark old version as superseded.
- Search knowledge base.
- View ingestion errors.
- View high-level usage/intent analytics.

Analytics

- Top intents.
- Languages used.
- Number of successful answers.
- Number of mobile handoffs.
- Most common unresolved questions.
- RAG retrieval failures.
- STT failure rate.
- Average response latency.
- No raw sensitive conversation data should be exposed by default.

14. Step-by-Step Development Phases

Phase 0 — Project setup

- Create Git repository and monorepo structure.
- Create backend, kiosk frontend, mobile frontend, and admin projects.
- Add .env configuration and secrets management.
- Set up database migrations.
- Create Docker Compose for local development if desired.
- Create a README with exact run commands.

Phase 1 — Build the kiosk shell

- Create portrait-first UI.
- Implement idle screen with chatbot emoji.
- Implement language selector.
- Implement TALK/HELP fallback button.
- Implement listening, processing, speaking, and error states.
- Run the frontend on the embedded smartphone browser or WebView.

Phase 2 — Backend session layer

- Implement /session.
- Implement session state and message storage.
- Implement /chat with a temporary mock response.
- Verify that kiosk sessions survive refresh/reconnect.

Phase 3 — Voice

- Add wake-word detection.
- Add microphone capture.
- Add STT.
- Add TTS.
- Add language detection/selection.
- Test noisy-room behavior.

Phase 4 — RAG

- Create document ingestion pipeline.
- Create embeddings/vector store.
- Add metadata filters.
- Add retrieval citations.
- Implement grounded response prompt.
- Add no-evidence fallback.

Phase 5 — Feature intents

- Cooperative governance.

- Scheme/service navigation.
- Agricultural support/insurance.
- Financial literacy.
- Grievance navigation.
- Document explanation.

15. Step-by-Step Development Phases — Continued

Phase 6 — Action-plan engine

- Define a structured ActionPlan schema.
- Generate steps only from retrieved evidence.
- Generate required-document checklist.
- Generate warnings/limitations.
- Attach sources to each major step where possible.

Phase 7 — QR handoff

- Implement signed session token.
- Create QR generator.
- Create mobile join endpoint.
- Build mobile session screen.
- Verify that the exact conversation context appears on mobile.
- Implement session expiry and invalid-token states.

Phase 8 — Image/document analysis

- Build mobile camera/upload component.
- Add image compression.
- Add OCR.
- Add document extraction.
- Add image understanding.
- Connect results to RAG.
- Show structured explanation and source references.

Phase 9 — Grievance workflow

- Classify complaint.
- Retrieve relevant escalation information.
- Generate complaint draft.
- Generate evidence checklist.
- QR handoff for long-form submission.

Phase 10 — Admin

- Document upload.
- Metadata editor.
- Approval workflow.
- Versioning.
- Search/index status.
- Basic analytics.

Phase 11 — Hardware enclosure

- Measure phone dimensions.
- Design/cut enclosure.
- Mount phone securely.
- Provide charging cable path.
- Add document stand.
- Add optional ESP32/button/LED.
- Test microphone and speaker openings.

Phase 12 — Integration and polish

- End-to-end testing.
- Latency optimization.
- Accessibility review.
- Language testing.
- Network failure handling.
- Demo data cleanup.
- Prepare scripted demo scenarios.

16. AI Development-Agent Instructions

Give the development AI the following implementation rules.

- Build the prototype incrementally; do not generate the entire system as one untestable code dump.
- After each module, create a runnable implementation and a short test procedure.
- Use typed schemas/models for API requests and AI outputs.
- Never hard-code government facts into prompts; store them in the knowledge base.
- Separate provider adapters so STT, TTS, LLM, OCR, and vector DB providers can be swapped.
- Keep kiosk and mobile clients thin; business logic belongs in the backend.
- Implement deterministic routing for speech-only versus mobile handoff.
- Return citations/source metadata with RAG answers.
- Log structured errors without storing unnecessary sensitive content.
- Provide seed documents and mock data for local testing.
- Provide one-command local startup where practical.
- Include automated tests for session creation, QR expiry, RAG retrieval, response-mode routing, and upload validation.

Suggested response schema

```
{
  "answer_text": "...",
  "spoken_text": "...",
  "response_mode": "SPEECH_ONLY | SCREEN_SUMMARY | MOBILE_HANDOFF | HUMAN_ESCALATION",
  "intent": "...",
  "action_plan": null,
  "sources": [],
  "qr": null,
  "language": "hi",
  "confidence": 0.0
}
```

Prompting pattern

System prompt should enforce: evidence-first answering, simple language, language consistency, explicit uncertainty, no invented facts, source attachment, response-mode classification, and safe escalation. User prompt should contain the transcript plus retrieved evidence and relevant session context.

17. Testing and Acceptance Criteria

Functional acceptance tests

Test	Expected result
Say 'Hello Sahayak'	Kiosk enters listening state.
Ask simple Hindi question	Hindi answer is spoken; no QR required.
Switch language	Next response uses selected language; context remains.
Ask complex question	Kiosk explains need for detailed view and shows QR.
Scan QR	Mobile opens and joins same session.
Ask follow-up on mobile	Backend retains previous context.
Upload document	File is validated, processed, and analyzed.
No evidence available	System states limitation instead of fabricating.
Invalid/expired QR	Mobile shows safe expired-session message.
Network interruption	UI gives retry/offline message without crashing.
Admin uploads document	Document remains pending until approved.
Supersede document	Old version is not treated as current.

Non-functional targets for demo

- Simple first interaction: user should know how to start without staff assistance.
- Low UI complexity and large touch targets.
- Graceful handling of speech recognition failures.
- No visible stack traces or technical errors.
- Consistent response language.
- Citations visible on mobile for substantive answers.
- QR handoff should take only a few user actions.
- Session continuity must be reliable.

18. Recommended Hackathon Demo Flow

Demo objective: prove that the physical box is useful by itself, while the personal phone acts as an extension rather than a replacement.

- 1 Idle: Judge sees chatbot emoji and 'Say Hello Sahayak'.
- 2 Wake: Say 'Hello Sahayak'. Device visibly changes to listening state.
- 3 Simple query: Ask a basic cooperative or financial-literacy question. Sahayak answers entirely through voice.
- 4 Problem query: Ask a more involved crop-insurance/scheme question. Sahayak asks only necessary clarifying questions and gives a short action summary.
- 5 Evidence: Show source references on screen/mobile dashboard for the answer.
- 6 Complex handoff: Ask for a detailed checklist or request image analysis. Sahayak explains that a phone is better for this task and displays QR.
- 7 Mobile continuation: Scan QR. The exact conversation appears without repeating the question.
- 8 Scan: Use the personal phone to capture a sample document/image.
- 9 Analyze: OCR + AI explains the document in the chosen language.
- 10 Grievance: Demonstrate complaint understanding and draft generation.
- 11 Return: Show that the kiosk can remain the accessible first point of interaction.

The strongest narrative

“Sahayak does not force every user onto a smartphone. It uses the kiosk for simple, immediate help and uses the user's phone only when the task genuinely needs richer interaction.”

19. Unique Selling Proposition

The differentiation should be framed around workflow completion, not simply chatbot intelligence.

USP pillar	What it means
Voice-first accessibility	User can speak naturally instead of typing.
Simple-language assistance	The system translates and simplifies, rather than merely translating words.
Verified answers	RAG answers are grounded in an approved knowledge base with source metadata.
Problem → Action	The system converts a question into practical next steps and document checklists.
Kiosk independence	Common questions are solved without requiring the user's personal smartphone.
Seamless QR extension	Complex tasks continue on the user's phone without restarting the conversation.
Scan & Understand	Real documents/images can become inputs for explanation and guidance.
Grievance workflow	The assistant helps structure complaints and navigate documented escalation routes.
Admin verification loop	Knowledge can be reviewed, versioned, verified, and superseded.

One-line USP

“Sahayak is a voice-first multilingual rural assistance kiosk that turns questions about cooperatives, schemes, agriculture, finance, and grievances into verified, understandable, step-by-step help—solving simple problems on the kiosk and seamlessly handing complex tasks to the user's own phone.”

20. Risks and Mitigations

Risk	Mitigation
LLM hallucination	Verified RAG, source citations, no-evidence fallback.
Outdated scheme/legal information	Admin verification date, versioning, superseded status.
Poor wake-word recognition	Local fallback TALK/HELP button.
Noisy environment	Short recording windows, VAD, repeat option, visible transcript.
Poor network	Cache static UI and selected approved content; show clear offline state.
QR session hijacking	Short-lived signed/random tokens, HTTPS, expiry, no PII in QR.
User privacy	Minimize data retention; explicit upload consent; secure storage.
Too much complexity	Keep kiosk UI minimal; route complex work to mobile.
Prototype hardware failure	Use smartphone as core device and keep backup phone/power.
Slow AI responses	Streaming where supported, concise prompts, retrieval before generation, loading state.
Legal overclaiming	Present as information/guidance; cite source and escalate ambiguous cases.

What not to build in the first prototype

- Do not add biometrics unless the problem specifically requires them.
- Do not add blockchain just for novelty.
- Do not build a custom LLM from scratch.
- Do not require a Raspberry Pi when the embedded smartphone can handle the kiosk workload.
- Do not build a native mobile app first if a PWA can demonstrate the QR flow.
- Do not load unverified internet content into the answer system.
- Do not make every question require QR handoff; the kiosk itself must visibly solve common tasks.

21. Final Prototype Deliverables

Deliverable	Definition of done
Physical Sahayak box	Embedded smartphone securely mounted, powered, visible, usable.
Kiosk UI	Idle/wake/listen/process/speak/error/QR states implemented.
Wake interaction	Hello Sahayak triggers listening, with TALK fallback.
Voice pipeline	STT + AI + TTS working end-to-end.
Multilingual flow	At least two languages demonstrated reliably.
RAG knowledge base	Approved documents indexed with metadata and citations.
Intent engine	Core problem categories routed correctly.
Action-plan engine	Step-by-step guidance + checklist for supported workflows.
Mobile PWA	QR opens session and continues conversation.
Image/document analysis	Phone capture → OCR/analysis → explanation.
Grievance flow	Complaint understanding + draft/checklist.
Admin console	Document upload/review/approval/versioning.
Testing suite	Core automated and manual acceptance tests.
Demo package	Sample documents, scripted queries, backup assets, README.

Recommended MVP order

1. Kiosk UI → 2. Session backend → 3. Voice → 4. RAG → 5. Core intents → 6. Action plans → 7. QR mobile handoff → 8. Image analysis → 9. Grievance → 10. Admin → 11. Hardware polish → 12. Testing/demo.

This order ensures that a working end-to-end prototype exists early. Every later feature improves the same core system instead of creating disconnected demonstrations.

End of implementation plan